

第11-12课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 11-12课 PDF 解析整理版

第 11课+第12 课：ChatBI前端体验层从"能跑通"到"真能用"

- 第11课（实时性升级）：在第10课已经跑通的同步 `FastAPI` 接口基础上，把"提问→等8秒→结果突然出现"的体验升级为"提问→SQL 逐字流出→结果到位"。后端引入 `LLMClient.stream + run_stream` 生成器 + `StreamingResponse` 三层改造，前端用最轻量的单 `HTML + EventSource` 把流式数据渲染出来。
- 第12课（完整性补齐）：在第11课"能跑通"的基础上，把"能用"补到"好用"——前端补对话卡片、SQL 一键复制、生成/总耗时、空结果友好提示；后端修复 `Decimal` 类型的 `JSON` 序列化崩溃；用 `FastAPI` 自动生成的 `OpenAPI` 文档导入 `Apifox`，把前后端接口约定落到纸面。

[链接] 逻辑衔接点

- 接口继承：第12课保留第11课新增的 `POST /api/v1/query/stream` `SSE` 端点不动，把同步接口 `/api/v1/query` 的"好用的部分"也补齐（如 `duration_ms`）。
- 链路串接：第12课的 `_json_serializer` 兜底直接作用在第11课 `_sse_event` 的 `json.dumps` 上，确保 `Decimal` 字段（如销售额、利润）能被正确序列化到 `SSE` 的 `result` 事件里。
- 体验层叠加：第12课的前端是在第11课 `SSE` 消费代码上增加对话卡片、复制按钮、耗时、空结果提示等节点，不替换第11课的 `sql_chunk / sql_done / result / error` 四类事件协议。
- 接口文档化：第12课把第11课新增的 `SSE` 端点的"事件类型表格"写进 `FastAPI description` 字段，再通过 `/openapi.json` 导入 `Apifox`，作为前端/后端/测试三方的统一接口约定。

两节课的关系 / 总结

维度

第 11 课（流式接口）

第 12 课（体验闭环）

所处阶段

第三阶段《系统服务化与前端交互》第二节

第三阶段《系统服务化与前端交互》最后一节

核心问题

同步接口等 5-8 秒全程无反馈，用户分不清"思考中"与"挂了"

链路能跑通 ≠ 体验闭环；缺历史、缺耗时、Decimal 崩、接口无文档

后端改造点

LLMClient stream / run_stream 生成器 / StreamingResponse 端点

Decimal JSON 序列化兜底 / Apifox 接入

前端改造点

单 HTML + EventSource 消费 SSE 流

对话卡片 / SQL 复制按钮 / 耗时 / 空结果提示

关键概念

SSE / StreamingResponse / ReadableStream / 事件协议

JSON 序列化兜底 / OpenAPI / 体验闭环

交付物

浏览器能看到 SQL 逐字流出

"能用"→"好用"+ 团队级接口规范

一句话关系总结：

- 第 11 课让用户"等得有反馈"（同步→流式，5-8 秒的等待过程被实时呈现）；
- 第 12 课让用户"用得沉淀"（历史/复制/耗时/空结果 + 接口协作规范把链路从 demo 变成可上线）；
- 第 12 课不替换第 11 课的 SSE 事件协议，而是在其上叠加前端节点 + 补后端 JSON 序列化兜底；最终通过 OpenAPI 文档把这条链路沉淀为团队级接口规范。

第 11 课打通了"链路能通"，第 12 课把链路补到"真的好用"。两节课都聚焦 ChatBI 的前端体验层，组合起来完成了第三阶段《系统服务化与前端交互》的最后一公里。

- 第 11 课 · SSE 流式接口与前端骨架：把第 10 课同步接口升级为 SSE 流式输出，前端从"对着空白页等 5-8 秒"变为"逐字看到 SQL 在生成"。后端三层改造（LLMClient 支持 stream → ChatBISystem.run_stream 生成器 → FastAPI StreamingResponse 端点），前端用最轻量的单 HTML 文件对接 SSE。

-	第	12	课 · 前后端联调与体验闭环：在第	11
课"能跑通"的基础上补齐"能用"到"好用"的差距——前端增加对话式历史卡片、SQL 一键复制、生成耗时与总耗时展示、空结果友好提示；后端修复 Decimal 类型 JSON 序列化崩溃的坑；通过 FastAPI 自动生成的 OpenAPI 文档导入 Apifox，把前后端接口约定落到纸面。				
一句话关系：第	11	课解决"实时性"，第	12	课解决"完整性"——前者让用户等过程，后者让用户等得有反馈、查得有沉淀、问题不会突然崩。
维度				
第 11 课				
第 12 课				
核心问题				
同步接口全程无反馈，用户分不清"思考中"与"挂了"				
链路能通 ≠ 体验闭环；缺历史、缺耗时、Decimal 崩、接口无文档				
后端改造点				
LLMClient 流式 / run_stream 生成器 / SSE 端点				
Decimal 序列化 / Apifox 接入				
前端改造点				
单 HTML + EventSource 消费流				
对话卡片 / 复制按钮 / 耗时 / 空结果				
关键概念				
SSE / StreamingResponse / ReadableStream / 事件协议				
JSON 序列化兜底 / OpenAPI / 体验闭环				
交付物				
浏览器能看到 SQL 逐字流出				
"能用"→"好用"+ 团队级接口规范				

第 11 课：SSE 流式接口与前端骨架

课程定位与目标

课程定位：

- 所处阶段：第三阶段《系统服务化与前端交互》第二节
- 本课核心问题：第 10 课已经把 ChatBI 封装成了 FastAPI 同步接口，但用户从提问到拿到结果要等 5-10 秒，全程面对空白页面——这个等待体验在生产环境不可接受

本节课目标：

- 理解为什么 ChatBI 必须做流式输出，同步接口在体验上的硬伤是什么
- 能将 LLMClient 改造为支持 stream=True 的流式调用
- 能在 ChatBISystem 中新增 run_stream() 生成器，按阶段 yield SSE 事件
- 能在 FastAPI 中新增 POST /api/v1/query/stream SSE 端点
- 能用最轻量的单 HTML 文件对接 SSE，实现 SQL 逐字流式渲染

开场引入

第 10 课结束时，ChatBI 已经有了两个可用接口：

GET /health	→ 服务和数据库是否在线
POST /api/v1/query	→ 输入问题，一次性返回 SQL + 结果

上节课用 curl 调一下同步接口：

```
curl -X POST http://127.0.0.1:8000/api/v1/query \  
-H "Content-Type: application/json" \  
-d '{"question": "查询上个月的利润"}'
```

此时我们会发现：

- 发出请求后，大约要等 5-8 秒——这段时间里，终端没有任何输出
- 如果换成浏览器，就是对着空白页面等了 5-8 秒，然后突然一下全部内容跳出来。

[要点] 同步接口的硬伤不是“慢”，而是全程没有反馈。用户分不清“系统在思考”还是“系统挂了”。

一、为什么 ChatBI 必须做流式

1. 两种交互模式的体验差异

维度

同步模式

流式模式

用户感知

等了 5 秒什么都没有，然后结果突然弹出

第 1 秒就能看到 SQL 开头，逐字流出来

心理状态

"是不是挂了？"

"系统正在思考"

放弃成本

5 秒后才知道要不要重试

随时能看到进展，随时决定是否等下去

2. 为什么传统 Web 应用不需要流式

一个"查询订单列表"的接口，数据库通常 50ms 就返回了——用户根本感知不到等待。

但 ChatBI 不一样：

- LLM 生成 SQL 需要 5-8 秒——这是模型推理的固有延迟，再怎么优化也低不到毫秒级
- 用户的问题是开放式分析型问题，不像点个菜单那么快
- LLM 本身就支持流式输出——OpenAI 的 `stream=True` 就是为此设计的

[要点] 流式不是"性能优化"，而是交互范式升级。同步接口让用户等结果，流式接口让用户等过程。

3. 流式的核心思路

把一次请求的完整生命周期，拆成几个可以逐步推送的阶段：

前端不再等一个完整 JSON，而是持续接收事件，实时更新页面。

二、后端改造第一层：LLMClient 支持流式

1. OpenAI SDK 的流式用法

llm_client.py 里的 generate_sql() 调用方式:

```
response = self.client.chat.completions.create(
    model=self.model,
    messages=[...],
    temperature=self.temperature,
    max_tokens=self.max_tokens
)
# 一次性拿到完整输出
raw_output = response.choices[0].message.content.strip()
```

流式只需要改两处:

- 加 stream=True
- 把 response.choices[0].message.content 改成遍历 chunk.choices[0].delta.content

2. 新增 generate_sql_stream 方法

llm_client.py 新增方法:

```
def generate_sql_stream(self, system_msg: str, prompt: str) -> Generator[str, None, None]:
    """调用 LLM 流式生成 SQL (逐 chunk 产出)"""
    stream = self.client.chat.completions.create(
        model=self.model,
        messages=[
            {"role": "system", "content": system_msg},
            {"role": "user", "content": prompt}
        ],
        temperature=self.temperature,
        max_tokens=self.max_tokens,
        stream=True,
    )

    for chunk in stream:
        delta_content = chunk.choices[0].delta.content
        if delta_content is not None:
            yield delta_content
```

三个关键点:

- stream=True 告诉 SDK 不要等完整响应, 而是逐 chunk 返回
- chunk.choices[0].delta.content 里是增量文本 (delta 不是完整内容, 而是"这次多出来的部分")
- delta_content is not None 判断必须加——最后一个 chunk 的 delta.content 是 None, 不判断会报错

[要点] 流式方法每次 yield 一小段文本, 调用方可以实时推送给前端。原有的 generate_sql 保留不动, 确保向后兼容。

三、后端改造第二层: ChatBISystem 的 run_stream

1. 先设计 SSE 事件协议

在写代码之前，先定义前后端的"契约"——SSE 事件类型：

事件类型

含义

数据字段

sql_chunk

LLM 流式产出的 SQL 片段

content（增量文本）

sql_done

SQL 完整输出

sql（清理后的完整 SQL）

result

查询结果

columns、rows、row_count

error

异常信息

error、error_type、sql（如有）

这个协议一旦定下来，前端就按这 4 种事件来处理，不用猜测服务端的行为。

2. SSE 事件格式

每个事件的标准格式：

```
event: sql_chunk
data: {"content": "SELECT"}
```

注意最后有一个空行——这是 SSE 协议要求的事件分隔符。

3. _sse_event 辅助函数

main.py 中新增辅助函数：

```
import json
import re
from typing import Generator

def _sse_event(event_type: str, data: dict) -> str:
    """构造 SSE 格式的事件字符串"""
    return f"event: {event_type}\ndata: {json.dumps(data, ensure_ascii=False)}\n\n"
```

这个函数的作用只有一个：把事件类型和数据打包成 SSE 格式的字符串，后续所有事件都通过它统一生成。

4. run_stream 方法主体

ChatBISystem 新增方法：

```
def run_stream(
    self,
    user_question: str,
    use_few_shot: bool = True,
    use_rules: bool = True,
    use_guards: bool = True,
    use_indicator_knowledge: bool = True
) -> Generator[str, None, None]:
    # 1. 解析问题
    parsed = self.parser.parse(user_question)
    if not self.parser.validate(parsed):
        yield _sse_event("error", {
            "error": "输入问题为空",
            "error_type": "validation"
        })
        return

    # 2. 构造 Prompt (同 run 方法)
    detected_indicators = []
    indicator_block = ""
    if use_indicator_knowledge:
        detected_indicators, indicator_block = self.indicator_knowledge.build_knowledge_block(user_question)

    system_msg, prompt = build_prompt(
        user_question,
        use_few_shot=use_few_shot,
        use_rules=use_rules,
        use_guards=use_guards,
        indicator_knowledge=indicator_block
    )

    # 3. 流式生成 SQL —— 逐 chunk 推送
    sql_parts = []
    try:
        for chunk_text in self.llm.generate_sql_stream(system_msg, prompt):
            sql_parts.append(chunk_text)
            yield _sse_event("sql_chunk", {"content": chunk_text})
    except Exception as e:
        yield _sse_event("error", {"error": str(e), "error_type": "llm"})
```

```

    return

# 4. 拼接完整 SQL 并清理 markdown 标记
raw_sql = "".join(sql_parts)
sql = re.sub(r'```\sql|```', '', raw_sql).strip()
yield _sse_event("sql_done", {"sql": sql})

# 5. 执行 SQL
try:
    columns, results = self.db.execute(sql)
    rows_dict = [dict(zip(columns, row)) for row in results]
    yield _sse_event("result", {
        "columns": columns,
        "rows": rows_dict,
        "row_count": len(results),
    })
except Exception as e:
    yield _sse_event("error", {
        "error": str(e),
        "error_type": "database",
        "sql": sql,
    })

```

这段代码和 `run()` 方法的最大区别在于：

- `run()` 把所有步骤走完再一次性返回——成功返回 `dict`，失败也返回 `dict`
- `run_stream()` 每走完一步就 `yield` 一个事件——前端可以逐步收到进展

[要点] `run_stream` 的设计原则：每个阶段都是独立的 `yield` 点。前端可以在任意时刻展示当前阶段的进展，不需要等全链路走完。

四、后端改造第三层：FastAPI SSE 端点

1. StreamingResponse 是什么

FastAPI 提供了 `StreamingResponse`，专门用于把生成器的输出作为 HTTP 响应流式推送给客户端：

```

from fastapi.responses import StreamingResponse

return StreamingResponse(
    event_generator(), # 一个 yield 字符串的生成器
    media_type="text/event-stream", # SSE 的标准 MIME 类型
)

```

只要生成器还在 `yield`，连接就一直保持；生成器结束，连接关闭。

2. 新增 SSE 端点

`api_service.py` 新增端点：

```
@app.post("/api/v1/query/stream")
async def query_chatbi_stream(payload: QueryRequest) -> StreamingResponse:
    def event_generator():
        for event_str in system.run_stream(
            user_question=payload.question,
            use_few_shot=payload.use_few_shot,
            use_rules=payload.use_rules,
            use_guards=payload.use_guards,
            use_indicator_knowledge=payload.use_indicator_knowledge,
        ):
            yield event_str

    return StreamingResponse(
        event_generator(),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no",
        },
    )
```

三个关键响应头：

- Cache-Control: no-cache: 禁止浏览器缓存 SSE 流
- Connection: keep-alive: 保持长连接
- X-Accel-Buffering: no: 如果前面有 nginx 反代，告诉它不要缓冲

五、前端骨架：单 HTML 文件对接 SSE

1. 为什么不引入前端框架

本课的前端只有一个目标：验证 SSE 链路能跑通。

React/Vue 等框架的工程化优势在组件复用和状态管理，但我们这里只需要：

- 一个输入框 + 一个按钮
- 一个 SQL 展示区
- 一个结果展示区

这些用原生 HTML + CSS + JS 就够了，引入框架反而是负担。

2. 前端核心流程

3. SSE 请求的核心代码

前端 JavaScript 核心逻辑：

```
const response = await fetch('/api/v1/query/stream', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ question }),
});

const reader = response.body.getReader();
const decoder = new TextDecoder();
let buffer = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;

  buffer += decoder.decode(value, { stream: true });

  // 按 \n\n 分割 SSE 事件
  const parts = buffer.split('\n\n');
  buffer = parts.pop(); // 最后一段可能不完整

  for (const part of parts) {
    let eventType = '';
    let eventData = {};

    for (const line of part.split('\n')) {
      if (line.startsWith('event: ')) {
        eventType = line.slice(7).trim();
      } else if (line.startsWith('data: ')) {
        eventData = JSON.parse(line.slice(6));
      }
    }

    switch (eventType) {
      case 'sql_chunk':
        // 逐字拼接 SQL
        fullSql += eventData.content;
        sqlBox.textContent = fullSql;
        break;
      case 'sql_done':
        fullSql = eventData.sql;
        sqlBox.textContent = fullSql;
        break;
      case 'result':
        renderResult(eventData);
        break;
      case 'error':
        renderError(eventData);
        break;
    }
  }
}
```

三个容易出错的地方：

- `buffer.split('\n\n')` 后要把最后一段留到下次——SSE 事件可能被 TCP 分片截断

- `line.startsWith('event:')` 解析事件类型，不是 `data:` 字段
- `eventData.content` 可能为空字符串，但不能当 `null` 处理

4. 静态文件挂载

让 FastAPI 直接托管前端 HTML 文件，避免单独启动前端服务器：

```
from fastapi.staticfiles import StaticFiles
from pathlib import Path

STATIC_DIR = Path(__file__).parent / "static"
if STATIC_DIR.is_dir():
    app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
```

这样访问 `http://localhost:8000/static/index.html` 就能打开前端页面。

5. CORS 中间件——前后端分离的必要配置

前端页面（`localhost:8000/static/index.html`）访问后端（`localhost:8000/api/v1/query/stream`），属于同源，不存在跨域问题。

但如果前端单独部署（比如 `file://` 协议或另一个端口），浏览器会拦截跨域请求。所以必须加 CORS 中间件：

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],          # 生产环境应限制为具体域名
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

[要点]

CORS

不是“要不要加”的问题，而是前后端分离架构下的必要配置。不加，浏览器直接拦截请求，连报错信息都看不到。

六、启动验证与效果演示

1. 环境准备

继续沿用 `uv` 管理环境，确认 `.env` 配置正确：

```
cd code/chatbi_mvp
uv sync
```

2. 启动服务

```
uv run uvicorn api_service:app --host 127.0.0.1 --port 8080
```

启动后，日志应显示：

```
Uvicorn running on http://127.0.0.1:8080  
Application startup complete.
```

3. 验证 1：健康检查

```
curl http://127.0.0.1:8080/health
```

预期返回：

```
{"status": "ok", "database_connected": true}
```

4. 验证 2：SSE 流式接口

```
curl -N -X POST http://127.0.0.1:8001/api/v1/query/stream \  
-H "Content-Type: application/json" \  
-d '{"question": "查询已完成订单的总数量"}'
```

预期看到逐行输出 SSE 事件：

```
event: sql_chunk  
data: {"content": "SELECT"}  
  
event: sql_chunk  
data: {"content": " COUNT"}  
  
event: sql_chunk  
data: {"content": "(*) FROM"}  
  
event: sql_done  
data: {"sql": "SELECT COUNT(*) FROM sales_orders WHERE order_status = 'completed';"}  
  
event: result  
data: {"columns": ["COUNT(*)"], "rows": [{"COUNT(*)": 150}], "row_count": 1}
```

如果看到 sql_chunk 逐个出现，说明流式链路已经打通。

5. 验证 3：浏览器打开前端

访问 <http://127.0.0.1:8080/static/index.html>，在输入框中输入“查询已完成订单的总数量”，点击“查询”。

预期效果：

- SQL 区域逐字流式输出
- 光标闪烁动画表示“正在生成”

- SQL 生成完毕后，下方出现结果表格

小结

本节课完成了从"同步接口"到"流式接口"的三层改造：

[要点]

本节课的核心不是"多了一个接口"，而是交互范式从"等结果"变成了"等过程"。用户不再对着空白页面等待，而是实时看到 SQL 在生成。

后端保留了两条接口并存：

- POST /api/v1/query — 同步接口，适合脚本和自动化调用
- POST /api/v1/query/stream — SSE 流式接口，适合前端交互

下一课的重点是把这个"链路能通"的系统补到"真的好用"——结果表格渲染、加载状态、错误提示、超时取消和端到端演示。

第 12 课：前后端联调与体验闭环

课程定位与目标

课程定位：

- 所处阶段：第三阶段《系统服务化与前端交互》最后一节
- 本课核心问题：第 11 课已经把 SSE 流式链路跑通了——前端能发请求，SQL 能逐字流出，结果能渲染成表格——但整个系统离"真的能用"还有几步：结果为空时页面没有友好提示、前端不知道查了多久、SQL 没法一键复制、前后端联调时接口定义没有统一文档、后端返回的 Decimal 类型会导致 JSON 序列化失败

本节课目标：

- 理解"链路能通"和"体验闭环"之间的差距——前者是技术验收，后者是产品验收
- 能在前端补齐查询历史记录、耗时展示、SQL 复制、空结果友好提示等体验闭环
- 理解并修复 Decimal 类型 JSON 序列化问题——这是真实联调中必然遇到的坑
- 能通过 /openapi.json 获取接口文档，导入 Apifox 实现前后端接口约定统一
- 完成第三阶段端到端演示，从自然语言提问到格式化结果返回的完整闭环

开场引入

第 11 课结束时，我们有了这条链路：

```
前端 HTML → fetch SSE → FastAPI /api/v1/query/stream → ChatBISystem.run_stream() → LLM 流式生成 SQL → DB 执行 → SSE 事件逐条推送
```

链路确实通了。但实际使用中，以下场景仍然存在明显的体验缺口：

- 1. 输入"利润"——LLM 要 8 秒才能吐完 SQL，但前端光标一直在闪，没有进度提示
- 2. SQL 生成了——想复制出来检查一下，没法一键复制
- 3. 结果返回了——但全是 Decimal 类型的数字，前端报 `TypeError: Object of type Decimal is not JSON serializable`，整个流式链路直接崩掉
- 4. 查一个不存在的数据——SQL 执行成功但 0 行结果，前端只显示一个空表格，用户不知道是"没有数据"还是"出 bug 了"
- 5. 想看上次查了什么——页面刷新后全没了，没有查询历史

本课把这些"能用但不好用"的地方逐一补齐。

一、修复 Decimal 序列化——联调的第一个坑

问题复现

启动第 11 课的代码，查一个涉及金额的问题：

```
curl -s -N -X POST http://127.0.0.1:8000/api/v1/query/stream \
-H "Content-Type: application/json" \
-d '{"question": "查询总销售额"}' --max-time 30
```

如果数据库的金额字段是 DECIMAL 类型（MySQL 中非常常见），后端会直接报错：

```
event: error
data: {"error": "Object of type Decimal is not JSON serializable", "error_type": "database"}
```

根因分析

`run_stream()` 里的 `_sse_event()` 使用 `json.dumps()` 序列化数据。MySQL 的 DECIMAL 类型在 Python 中是 `decimal.Decimal` 对象，而 `json.dumps` 默认不知道如何处理它。

修复方案

在 `main.py` 中新增一个 JSON 序列化补充函数：

```
import json
```

```

from decimal import Decimal
from typing import Generator

def _json_serializer(obj):
    """JSON 序列化补充：处理 Decimal 等非标准类型"""
    if isinstance(obj, Decimal):
        return float(obj)
    raise TypeError(f"Object of type {type(obj)} is not JSON serializable")

def _sse_event(event_type: str, data: dict) -> str:
    """构造 SSE 格式的事件字符串"""
    return f"event: {event_type}\ndata: {json.dumps(data, ensure_ascii=False, default=_json_serializer)}\n\n"

```

关键改动：`json.dumps(data, default=\_json\_serializer)` —— `default` 参数是一个回调函数，当 `json.dumps` 遇到不认识的类型时，会调用它来兜底。

验证修复

再次 curl 查询金额：

```

curl -s -N -X POST http://127.0.0.1:8000/api/v1/query/stream \
-H "Content-Type: application/json" \
-d '{"question": "查询总销售额"}' --max-time 30

```

现在能正确返回：

```

event: result
data: {"columns": ["total_amount"], "rows": [{"total_amount": 19915.0}], "row_count": 1, ...}

```

Decimal(19915.00) 被正确转为 19915.0。

二、前端体验闭环——从“能用”到“好用”

2.1 对话式历史记录

第

11

课的前端只有“一次查询”的能力，刷新就没了。改为“对话卡片”结构：每次查询生成一张卡片，新查询插入到最上方，形成对话历史。

改动要点：

- 移除 `.result-area` 单区域设计，改为 `.chat-history` + `.chat-card` 列表
- 每张卡片包含：问题行（Q 图标 + 问题文本 + 时间）、SQL 区域、结果区域
- 新查询 `insertBefore(card, chatArea.firstChild)` 实现最新在上

```

// 创建对话卡片
const card = document.createElement('div');
card.className = 'chat-card';

```

```
// 问题行
const questionRow = document.createElement('div');
questionRow.className = 'question-row';
questionRow.innerHTML = `
  <div class="q-icon">Q</div>
  <div class="q-text">${escapeHtml(question)}</div>
  <div class="q-time">${new Date().toLocaleTimeString('zh-CN', {hour:'2-digit', minute:'2-digit'})}</div>
`;
card.appendChild(questionRow);

// ... SQL 区域、状态行、结果区域 ...

chatArea.insertBefore(card, chatArea.firstChild); // 最新在上
```

2.2 SQL 复制按钮

SQL 生成完毕后，在 SQL 代码块下方出现"复制 SQL"按钮：

```
// SQL 工具栏
const sqlToolbar = document.createElement('div');
sqlToolbar.className = 'sql-toolbar';
sqlToolbar.style.display = 'none'; // 初始隐藏，sql_done 后才显示

const copyBtn = document.createElement('button');
copyBtn.className = 'copy-btn';
copyBtn.textContent = '复制 SQL';

// sql_done 事件触发后显示
copyBtn.onclick = () => {
  navigator.clipboard.writeText(fullSql).then(() => {
    copyBtn.textContent = '已复制';
    copyBtn.classList.add('copied');
    setTimeout(() => {
      copyBtn.textContent = '复制 SQL';
      copyBtn.classList.remove('copied');
    }, 2000);
  });
};
```

关键点：

- navigator.clipboard.writeText() 是浏览器原生 API，无需引入任何库
- 复制成功后按钮变绿 2 秒，提供即时反馈
- 工具栏只在 sql_done 事件后才显示，避免在 SQL 还在流式生成时出现

2.3 耗时展示

在 SQL 工具栏右侧显示耗时，区分"SQL 生成耗时"和"查询总耗时"：

```
const startTime = now(); // 发出请求的时刻
```

```
// sql_done 事件时
sqlEndTime = now();
durationSpan.textContent = `生成耗时 ${formatTime(sqlEndTime - sqlStartTime)}`;

// result 事件时
const totalTime = now() - startTime;
statusDiv.textContent = `查询完成，总耗时 ${formatTime(totalTime)}`;
```

formatTime 函数将毫秒转为人类可读格式：

```
function formatTime(ms) {
  if (ms < 1000) return `${Math.round(ms)}ms`;
  return `${(ms / 1000).toFixed(1)}s`;
}
```

2.4 空结果友好提示

SQL 执行成功但返回 0 行时，不做“空表格”，改为友好提示：

```
function renderResult(container, data) {
  const { columns, rows, row_count } = data;

  // 空结果友好提示
  if (!rows || rows.length === 0) {
    container.innerHTML = `
      <div class="empty-result">
        <div class="icon">&#128270;</div>
        <div>查询成功，但未找到匹配的数据</div>
      </div>
    `;
    return;
  }
  // ... 正常渲染表格 ...
}
```

2.5 过滤空 sql_chunk

部分模型在流式输出中会返回大量 content 为空字符串的 delta。在前端也加一层过滤：

```
case 'sql_chunk':
  // 过滤空内容
  if (!eventData.content) break;
  fullSql += eventData.content;
  sqlBox.textContent = fullSql;
  sqlBox.appendChild(cursorSpan);
  break;
```

后端同样过滤（在 main.py 的 run_stream 中）：

```
for chunk_text in self.llm.generate_sql_stream(system_msg, prompt):
  if not chunk_text:
    continue # 跳过空 chunk
  sql_parts.append(chunk_text)
  yield _sse_event("sql_chunk", {"content": chunk_text})
```

三、接口文档与 Apifox 联调

3.1 为什么要统一接口文档

前后端分离开发时，最常见的协作问题是：

- 后端改了字段名，前端不知道
- 前端传的参数格式不对，后端没校验
- SSE 的事件类型，双方理解不一致

这些问题在没有统一文档的情况下，只能靠口头沟通或翻代码，效率极低。

3.2 FastAPI 自带的 OpenAPI 支持

FastAPI 基于 Pydantic 的类型声明，自动生成了 OpenAPI 3.1.0 规范的接口文档：

```
app = FastAPI(  
    title="ChatBI MVP API",  
    version="0.2.0",  
    description=""...", # 支持 Markdown  
    openapi_tags=[  
        {"name": "查询", "description": "自然语言转 SQL 查询接口"},  
        {"name": "系统", "description": "系统运维与监控接口"},  
    ],  
)
```

给每个路由加上 tags 和 summary：

```
@app.post("/api/v1/query/stream", tags=["查询"], summary="SSE 流式查询（逐步推送）")  
async def query_chatbi_stream(payload: QueryRequest) -> StreamingResponse:  
    ...  
  
@app.get("/health", response_model=HealthResponse, tags=["系统"])  
def health_check() -> HealthResponse:  
    ...
```

启动服务后，直接访问：

```
# 获取 OpenAPI JSON  
curl http://localhost:8000/openapi.json  
  
# 或浏览器打开 Swagger UI  
http://localhost:8000/docs
```

3.3 导入 Apifox 实现前后端约定

Apifox 是国内主流的 API 协作工具，支持从 OpenAPI 导入接口定义：

导入步骤：

1. 启动 ChatBI 服务: `python -m uvicorn api_service:app --port 8000`
2. 在浏览器确认 OpenAPI 可访问: `http://localhost:8000/openapi.json`
3. 打开 Apifox → 选择项目 → 点击「导入」
4. 选择「URL 导入」→ 粘贴 `http://localhost:8000/openapi.json`
5. Apifox 会自动解析所有接口、请求体、响应体
6. 前端同事可以在 Apifox 中查看每个接口的字段定义、示例值

对于 SSE 流式接口的约定:

在 description 中我们详细列出了 SSE 事件类型表格, Apifox 会直接渲染出来:

事件类型

说明

data 结构

sql_chunk

SQL 文本片段

```
{"content": "SELECT... "}
```

sql_done

SQL 完整输出

```
{"sql": "SELECT COUNT(*)... "}
```

result

查询结果

```
{"columns": [...], "rows": [...], "row_count": N}
```

error

异常信息

```
{"error": "...", "error_type": "... "}
```

四、端到端执行

4.1 简单查询——快速出结果

输入：查询已完成订单的总数量

观察：

- 状态行先显示"正在生成 SQL..."（带旋转动画）
- SQL 逐字流出
- sql_done 后工具栏出现"复制 SQL"按钮和"生成耗时 8.2s"
- 状态行切换为"SQL 执行中..."
- 结果表格出现，状态行显示"查询完成，总耗时 8.5s"

4.2 涉及金额的查询——Decimal 修复验证

输入：查询总销售额

观察：

- 结果中的金额字段正确显示为浮点数（如 19915.0），不再报序列化错误
- Decimal 修复生效

4.3 空结果——友好提示验证

输入：查询2025年12月的订单数量

观察：

- SQL 执行成功但返回 0 行
- 前端展示"[检索] 查询成功，但未找到匹配的数据"，而不是一个空表格

4.4 SQL 复制

点击"复制 SQL"按钮，按钮变绿显示"已复制"，2 秒后恢复。粘贴到文本编辑器中确认是完整的 SQL。

4.5 接口文档查看

1. 浏览器打开 <http://localhost:8000/docs>，展示 Swagger UI
2. `curl http://localhost:8000/openapi.json` 展示 OpenAPI JSON
3. 在 Apifox 中导入，展示接口分组和 SSE 事件类型文档

五、第三阶段总结

第三阶段交付物回顾

课次

交付物

核心能力

第 10 课

FastAPI 同步服务

命令行脚本 → REST API

第 11 课

SSE 流式接口 + 前端骨架

等待 5-8 秒 → SQL 逐字流出

第 12 课

前端体验闭环 + 接口文档

"能用" → "好用"

技术栈演进

第 9 课: Python 命令行脚本 (纯后端)

↓

第 10 课: FastAPI + Pydantic + Uvicorn (HTTP 服务化)

↓

第 11 课: SSE + StreamingResponse + ReadableStream (流式链路)

↓

第 12 课: 对话式前端 + OpenAPI + Apifox (体验闭环 + 协作规范)

当前系统的局限

第三阶段结束，我们有了一个"能用的"ChatBI 系统，但：

- SQL 准确率不够：复杂查询（多表 JOIN、嵌套子查询、窗口函数）经常生成错误
- Schema 太大时 Prompt 撑爆：表多字段多时，DDL 就占了几千 token，LLM 注意力分散
- 没有上下文记忆：每次查询独立，不支持"上个月呢？"这种追问

这些正是第四阶段要解决的——Schema Linking + RAG + 多轮对话。

课后练习

建议 1、2、3 必做，4、5 选做：

1. 在前端添加一个"清空历史"按钮, 点击后清除所有对话卡片
2. 给同步接口 `/api/v1/query` 的响应也加上 `duration_ms` (对比流式接口的耗时差异)
3. 在 Apifox 中导入项目的 OpenAPI 文档, 创建一个 Mock 环境
4. 给 SSE 接口添加心跳机制: 每隔 15 秒推送一个 `event: ping\ndata: {}\n\n`, 防止 Nginx/CDN 因为长时间无数据而断开连接
5. 将 `json.dumps` 替换为 `orjson`, 对比序列化性能提升

附件 (飞书无下载权限): 第 11 课: SSE 流式接口与前端骨架.pdf

附件 (飞书无下载权限): 第 12 课: 前后端联调与体验闭环.pdf